# Lab 01: Correlation and Regression

Author: **N.J. de Winter** (*n.j.de.winter@vu.nl*)
Assitant Professor Vrije Universiteit Amsterdam
Statistics and Data Analysis Course

## Learning goals:

- Get famliar with Python through Jupyter
- Understand and apply tools to assess whether two variables are *correlated*
- Learn how to apply *regression* analysis to determine the shape of the *relationship* between two variables:
  - Simple linear regression (straight line relationship)
  - Polynomial regression (curved line relationship)
  - Exponental regression (exponentially growing or decaying relationship)

## Introduction

In this lab assignment, you will experiment with the tools in Python used to test correlation and regression. We will start with simple correlation, linear and non-linear regression and work towards multiple and logistic regression in the following Lab assignment.

This notebook format makes it possible to include live code in between the text for this assignment. Note that you can also make the assignment by copying the code from the code blocks (marked by `In []`) into your own Python environment (such as Spyder) and run the code there (shortcut: **CTRL + ENTER** on Windows/Linux and **CMD + ENTER** on Mac). Working in this Jupyter notebook probably helps you to keep your code more organized.

We start each session of Python (and Jupyter) by importing the *packages* we need. We will also use the command `%matplotlib inline` to make it possible to add plots in between our text output. To do so, run the code in the cell below:

In [ ]:
```
%matplotlib inline
import pandas as pd # The 'pandas' package helps us to import and manage data
import scipy.stats as stats # The 'scipy' package contains statistical formulas we
import statsmodels.formula.api as smf # The 'statsmodels' package contains the fun
from matplotlib import pyplot as plt # The 'matplotlib' package contains tools nee
```

Running the code above should not immediately yield a visible result, but it loads the functions you will need later into Python. If done correctly, you should see `In [1]` left of the cell above ( `In [*]` means the kernel is still processing your code). Note also that the text behind the `#` sign is not part of the code, but lists comments that show what the code does. This is a very handy way to keep your code organized and make sure you remember later on what your code does!

Now we will load the dataset you will need for this exercise using the code below. Note that for this to work, you will need to place the datafile ( `Lab01a.csv` ) in the same folder as this notebook.

In [ ]:
```
df = pd.read_csv('Lab01a.csv') # Load the data for this assignment into Python and
```

If you run the code of this notebook in your own Python environment, you first need to specify the working directory and then load the data using a slightly different command. The statement `'<Direction to folder containing dataset>'` should be replaced with the directory of the folder on your PC, e.g.: `C:/MyName/Documents/Statistics_and_Data_Analysis/Lab01/` .

WARNING: The code below does not need to be run if you are working in Jupyter.

In [ ]:
```
wdir = '<Direction to folder containing dataset>' # Set your working directory to
df = pd.read_csv(wdir + 'Lab01a.csv') # Load the data for this assignment from you
```

Now let's visualize our dataset by showing the first couple of rows:

In [ ]:
```
df.head()
```

The dataset you are looking at contains information about the worldwide monthly temperature anomaly (in degrees C) of the period 1979-2022 relative to the mean global temperature during the period 1951-1980 ( `Temperature_anomaly` column). It also lists the concentration of CO2 in the atmosphere measured monthly during this period (in parts per million by volume, or ppmV; `pCO2` column). The first column in your dataset ( `Date` ) lists the date in dd/mm/yyyy format and the second column numbers the months for easy plotting.

This dataset is obtained from the website **Our World in Data**, which is an excellent source of up-to-date information about climate, food, economic development, biodiversity and other pressing societal issues.

**Question 1:** Now that you have seen this dataset, can you think of a few questions we could solve by applying correlation and/or regression analyses on this data?

**Answer 1:** bla

# Part 1: Correlation

For starters, we might be interested in the correlation between atmospheric CO2 concentrations and global temperature.

**Question 2:** Which metric can help us to determine whether there is a correlation between these two variables?

**Answer 2:**

Let's try to calculate the Pearson's correlation coefficient between `pCO2` and the `Temperature_anomaly` . We can do thiss with the function `.corr` applied on the two

columns in the dataset `df` as follows:

In [ ]:
```
# Calculate Pearson's correlation between pCO2 and temperature anomaly
df['pCO2'].corr(df['Temperature_anomaly'], method = 'pearson')
```

**Question 3:** What do you think of this result? Is there a positive or a negative correlation between these two variables? Do you think the correlation is strong?

**Answer 3:**

Have a good look at the code cell above which you used to calculate the Pearson's correlation coefficient to make sure you understand how it works and what it does. You will be calculating more correlation coefficients in this and later assignments!

Ïf you are not sure what is going on here, it can be helpful to look at the documentation for the function ( `corr` in the pandas, or `pd` package). You can do that by using the `help()` function:

In [ ]:
```
# Print the help page for the "corr"-function in the pandas library ("pd")
help(pd.DataFrame.corr)
```

That's a lot of information, and much of it may be hard to understand at this stage. Don't panic, the most important thing is that you know how to use this function and how to interpret the output.

**Exercise 1:** To test yourself, you can apply the same function as above to see if there are positive or negative trends in `pCO2` and `Temperature_anomaly` with time in our dataset. In other words: You can test whether there are correlations between the `Month_since_January_1979` variable and either the `pCO2` and `Temperature_anomaly` variables in the dataset. Use the code cell below to do so and answer **Question 3** for the results of these two correlations as well.

In [ ]:
```
# Calculate Pearson's correlation between date (in months) and temperature anomaly


# Calculate Pearson's correlation between date (in months) and pCO2
```

**Answer 3 (for the new correlations):**

**Bonus exercise:** If you payed close attention to the `help()` output for the `corr` function, you might have spotted that there are multiple ways ( `methods` ) to calculate correlation between variables. If you are curious, you can try out some of the other correlation methods, such as `kendall` and `spearman` on the pairs of variables in our dataset using the empty code cell below. Is the result very different? What does this tell you about the correlation we found?

In [ ]:
```
# Bonus exercise: Try calculating correlation coefficients between variables in df
```

# Part 2: Simple linear regression

Now that we have discovered some correlations in our dataset, it is time to properly explore our dataset to verify if our correlations are not messed up by *outliers* or if the *form* of the relationship between our variables is not misleading us. We can do this by creating *scatter plots* of our data.

An example of a scatter plot showing the relationship between the temperature anomaly and time (in months) is produced by the code below. You can run it using the familiar shortcut (**CTRL + ENTER** on Windows/Linux and **CMD + ENTER** on Mac):

In [ ]:
```
# Plot temperature anomaly against time
plt.scatter(df.Month_since_january_1979, df.Temperature_anomaly, color = 'red', ma
plt.xlabel('# months since January 1979')
plt.ylabel('Temperature anomaly (in degrees C)')
```

Carefully read the code in the cell above and verify that you understand what it does. You will do much more plotting like this in future assignments!

Remember, if you are confused about a function (such as `plt.scatter`), you can use the `help()` function to look up its documentation and find out what it does:

In [ ]:
```
help(plt.scatter)
```

**Exercise 2:** To test whether you understand how to plot *scatter plots*, create plots of pCO2 vs time and pCO2 vs Temperature anomany in the two code cells below by modifying the code in the cell above used to plot Temperature anomaly vs time. Differentiate these scatter plots from each other by changing plot aesthetics such as the type of plotting symbol ( `marker` ), the color of the symbols ( `col` ) or the shading of the symbol ( `alpha` ). Use the help statement you ran in the previous code cell to guide your coding.

In [ ]:
```
# Plot pCO2 against time
```

In [ ]:
```
# Plot pCO2 against Temperature anomaly
```

We will now explore the shape of the relationship between pCO2 and Temperature using our dataset. To do so, we can run a *simple linear regression* using the **ordinary least squares (OLS)** algorithm we have discussed in the lectures. We will use the built-in function `smf.ols()` from the `statsmodels` package we loaded at the beginning of the assignment to do this:

In [ ]:
```
regression1 = smf.ols(formula = "df.Temperature_anomaly ~ df.pCO2", data = df).fit
```

Another new function! Make sure you understand what this one does as well. You know by now how to search for its documentation if you feel lost.

The object `regression1` contains all the information about our regression model. Let's print the regression coefficients:

In [ ]:
```
print('The regression coefficients are:\n', regression1.params)
```

The `\n` in the code above is used to create a new line, very handy when printing output!

**Question 4:** What is the meaning of the slope and intercept in this context?

**Answer 4:**

We can now use these coefficients to plot the regression line on top of our data:

In [ ]:
```
# Plot the result of simple linear regression between pCO2 and temperature
plt.scatter(df.pCO2, df.Temperature_anomaly, color = 'red', marker = '+')
plt.xlabel('Atmospheric pCO2 (ppmV)')
plt.ylabel('Temperature anomany (degrees C)')
plt.plot(df.pCO2, regression1.params[0] + regression1.params[1] * df.pCO2, color =
```

**Qeustion 5:** What do you think of this result? Does the regression line capture the relationship between the variables well?

**Answer 5:**

To test our relationship, we can calculate the F-value and p-value of the regression. Python can do this for us with one simple command:

In [ ]:
```
print(regression1.summary())
```

OK, that's a lot of information... Python just calculated a whole bunch of statistical metrics for this regression. The ones we are interested in are the $R^2$ value ( `R-squared` ), the F-value ( `F-statistic` ) and the p-value (or 'probability': `Prob (F-statistic)` ). You can also find the slope and intercept in this overview, as well as statistics on how certain we are about them (see `std err` values).

**Question 6:** At a 95% significance level (alpha = 0.05), is this linear relationship statistically significant?

**Answer 6:**

**Question 7:** How much of the variance in our pCO2 vs Temperature_anomaly dataset is explained by this linear relationship?

**Answer 7:**

**Question 8:** Based on our dataset over the period 1979-2022 and our simple linear regression, how much global warming would you expect if pCO2 increases by 100 ppmV? Can you give an uncertainty for this estimate? (Hint: use the standard error, or `std err` on the relevant regression coefficient)

**Answer 8:**

**Question 9:** The **Intergovernmental Panel on Climate Change (IPCC)** considers various **Shared Socioeconomic Pathways** in their projections for future climate. One of the more moderate scenarios (SSP2-4.5) predicts an atmospheric CO2 concentration of about 500 ppmV in the year 2100. Using the relationship you created in this exercise, what would you expect the temperature anomaly (global warming relative to the period 1951-1980) to be by

the end of this century? Is this a realistic estimate? Can you see any problems with this way of projecting future climate?

**Answer 9:**

# Part 3: Polynomial regression

To more thoroughly explore the trends in greenhouse gas concentrations and temperature on Earth, we will zoom out a bit and look at a dataset that goes further back in time.

**Exercise 3:** Use the code cell below to load this new dataset, which is called `Lab01b.csv`. You can reuse the code you used at the beginning of the assignment to load `Lab01a.csv`.

To prevent confusion between these two datasets, you will need to assign this new dataset a different name from the old one (which was called `df`). We will (rather uncreatively) go with the name `df2` for the remainder of this exercise, but you can give it another name if you like. Just remember that you need to be consistent with your names, otherwise Python will not understand to which object or dataset you are referring to.

In [ ]:
```
df2 = pd.read_csv('Lab01b.csv') # Load the second dataset for this assignment in th
```

This dataset contains annual average values of `pCO2` (atmospheric CO2 concentrations; in ppmV), `pCH4` (methane concentration; in ppb), `pN2O` (nitrous oxide concentration; in ppb) and `SST` (sea surface temperature anomalies; in degrees C relative to 1951-1980).

**Exercise 4:** You can inspect this dataset using the `head()` or `print()` commands you used before. Do so using the code cell below.

In [ ]:
```
# Inspect the new dataset
```

**Question 10:** What is striking about this dataset?

**Answer 10:**

CO2, CH4 and N2O are all greenhouse gases, so we expect them to covary with temperature.

**Exercise 5:** Use the code cell below to calculate the Pearson's correlation coefficient between each of the greenhouse gases and temperature for the time period 1850-2022. You can use the `corr` function and the code you used in Part 1 of this assignment.

Do you think the presence of `NaN` values poses a problem? Hint: Check the `help()` entry for the `corr` function you printed above to check this. It is good to be aware of empty values in your dataset, as they can have a big impact on your data analysis!

In [ ]:
```
# Calculate Pearson's correlation between pCO2 and temperature anomaly


# Calculate Pearson's correlation between pCH4 and temperature anomaly
```

```
# Calculate Pearson's correlation between pN2O and temperature anomaly
```

**Question 11:** Which of these greenhouse gas concentrations show a positive correlation with sea surface temperature? Is there a strong correlation?

**Answer 11:**

We will focus on the CO2 concentration now, and specifically on its trend through time.

**Exercise 6:** Use your newly gained experience with Python (and the code examples above) to create a plot of pCO2 with time ( `Year` ) in the code cell below.

In [ ]:
```
# Plot pCO2 vs time
```

**Exercise 7:** Repeat the steps in Part 2 of this assignment to calculate a linear regression through the pCO2 time trend and plot the regression result on top of the pCO2 dataset. Use the code cell below.

Think carefully about how you name the new regression object! Remember that Python cannot remember two objects of the same name, and make sure you give a new object a name that allows you to remember what is stored in the object.

In [ ]:
```
# Create linear regression between pCO2 and Year in dataset df2

# Print the regression coefficients of the new linear regression

# Plot pCO2 vs time with the linear regression line on top of the data

# Print the regression summary to check the strength and the significance of the l
```

**Question 12:** Do you think the linear regression approximates the form of the dataset well? What do the $R^2$, F and p-value statistics tell you?

**Answer 12:**

The exercise above shows you how important it is to look at scatter plots of your data instead of blindly executing statistical tests. I think you would agree that we can think of better models to approximate the rise of pCO2 since the Industrial Revolution.

We will try a **polynomial regression** to better approximate our pCO2 data. For this to work, we need to import some additional functions from the `sklearn` package, which we will need to preprocess our data for a polynomial regression. You can do so by running the code below.

In [ ]:
```
from sklearn.preprocessing import PolynomialFeatures # Import functions needed for
```

We can use the same OLS function as we used for linear regression to do a polynomial regression, but we need to pre-process our data first. For this we use the `PolynomialFeatures` function we just loaded. We will start by approximating the pCO2 data with a second degree polynomial function. Such a function contains a constant, a first order term and a second order term, yielding a function of the form:

$(y = a + b1 * x + b2 * x^2)$

Our OLS model will estimate values for `a`, `b1` and `b2`, but we will first transform our independent variable (in this case `Year`) to let Python know we will fit a polynomial function. Use the code below to do this, and inspect the resulting object.

In [ ]:
```
x2 = PolynomialFeatures(2, include_bias = False).fit_transform(df2.Year.values.resh

print(x2) # Inspect the resulting dataframe
```

Make sure you understand what happened here. You can always use the `help()` function to understand what `PolynomialFeatures` does, or Google it.

We will now perform the polynomial regression. This works much like the linear regression, but with the newly created `x2` as a set of independent variables:

In [ ]:
```
polyreg1 = smf.ols('df2.pCO2 ~ x2', data = df2).fit() # Regression of second degree

print(polyreg1.summary())
```

**Question 13:** How do the statistics of this polynomial model compare to those of the linear model you calculated before? Which model better describes the data? Which statistical metrics do you use to compare between them?

**Answer 13:**

Test the interpretations you made of the model output in the question above by plotting the result of the polynomial regression on top of the data. You can use the code in the cell below.

Note how similar this code is to the code you used to plot the linear regression results earlier in this exercise, and verify that you understand how this code works.

In [ ]:
```
# Plot pCO2 vs time with the second degree polynomial regression line on top of the
plt.scatter(df2.Year, df2.pCO2, color = 'red', marker = '+')
plt.xlabel('Year')
plt.ylabel('pCO2 (ppmV)')
plt.plot(df2.Year, polyreg1.params[0] + polyreg1.params[1] * df2.Year + polyreg1.pa
```

**Exercise 8:** I think we all agree that this result is an improvement, but maybe we can do better. Repeat the process you went through to estimate and plot the second order polynomial regression through the pCO2 dataset, but now create third and fourth order polynomials. Print the statistics for these models and plot them all together on top of the pCO2 data. Again: watch out how you name your objects and think carefully before you copy and paste code. Use the code cell below:

In [ ]:
```
# Transform the independent variable Year for a third and fourth degree polynomial

# Create regressions for the third and fourth degree polynomials

# Print summaries of third and fourth degree polynomial models

# Plot all polynomial models on the data in one plot
```

**Question 14:** What do you think of the result? Do the added polynomial terms improve the model? If so, do you think adding even more terms would be useful here? Can you find an optimum number of polynomial terms to accurately describe the shape of the pCO2 data while preventing overfitting?

**Answer 14:**

**Question 15:** After what we have learned from looking at a longer timeseries of pCO2, are you still equally confident about your answers to **Question 8 & 9**? If not, what changed your mind?

**Answer 15:**

# Part 4: Exponential regression

We will now briefly look at the pCO2 trend since 1850 using another type of regression model: exponential regression. We have discussed this type of regression in the lectures, but as a reminder you will find the general function for exponential regression models below:

$(y = c * e^{bx})$

However, we need to transform this function into its linear form to be able to use OLS. We can do this by taking the natural logarithm on both sides of the equation:

$(log(y) = log(c) + b * x)$

Now this is just a simple linear regression of the form $(y = a + bx)$, but with $y$ replaced by $log(y)$ and $a$ by $log(c)$. To use it, we first need to transform our dependent variable (pCO2) to its natural logarithm. For that, we need to import the `log()` function from the `numpy` package. After the transformation of our pCO2 variable, we can apply the OLS function to perform the linear regression. This code should look quite familiar to you now.

In [ ]:
```
import numpy as np # Import the numpy package that contains the log-function

logCO2 = np.log(df2.pCO2) # Create a new variable that is the natural logarithm of

expreg1 = smf.ols(formula = "logCO2 ~ df2.Year", data = df2).fit() # Perform the '

print(expreg1.summary()) # Print the regression summary
```

**Question 16:** Based on the statistics of the regression model, how well did the exponential regression perform? How does it compare to the linear and polynomial regressions you created before?

**Answer 16:**

**Exercise 9:** From the regression results, you can now calculate the parameters of the exponental function ($(y = c * e^{bx})$). Use the code cell below to calculate the parameters and plot the result of the exponential regression on top of the pCO2 data.

Tip: The `numpy` package contains the function `np.exp` which can be used to calculate $e^x$ to convert back from the natural logarithm

In [ ]:
```
# Calculate the parameter c by taking the natural exponent

# Plot the exponential model on top of the pCO2 data
```

**Question 17:** What do you think of the result? Can you think of a problem with our exponential model when trying to fit it to this type of data?

**Answer 17:**

**Bonus exercise:** We can probably make our exponential model fit better by modifying our variables `Year` and `pCO2` to relative values. We can do this by subtracting the minimum value in the dataset from all datapoints for both parameters. The function `np.min()` can help us with that:

In [ ]:
```
# Create relative pCO2 and Year variables
pCO2min = np.min(df2.pCO2) # find the minimum pCO2 value
Yearmin = np.min(df2.Year) # find the minimum Year (1850)

# Print the results
print('The minimum pCO2 value is', pCO2min, 'ppmV')
print('The starting Year is', Yearmin, 'AD')

pCO2min = 280 # Round the minimum pCO2 down to prevent zeroes in the dataset (log(0

pCO2rel = df2.pCO2 - pCO2min
Yearrel = df2.Year - Yearmin
```

Now repeat the exponential regression you performed above with these modified variables and assess whether it performs better.

In [ ]:
```
# Create a new variable that is the natural logarithm of the modified pCO2 variable

# Perform the exponential regression in its linearized form

# Print the regression summary

# Calculate the parameter c by taking the natural exponent

# Plot the exponential model on top of the pCO2 data
```